

Pre-Deployment Evaluation of LLM-based Conversational Agents through a Digital Twin-Inspired Approach

Caio Vinícius Cunha Bernardo
University of São Paulo (EACH-USP)
São Paulo, Brazil
caiovb@usp.br

Thais Rodrigues Neubauer
University of São Paulo (IEA-USP)
São Paulo, Brazil
thais.neubauer@usp.br

Gregório A. de Almeida Filho
University of São Paulo (EACH-USP)
São Paulo, Brazil
gregassagraf@usp.br

Enio Alterman Blay
University of São Paulo (IEA-USP)
São Paulo, Brazil
enio.alterman.blay@usp.br

Sarajane Marques Peres
University of São Paulo (EACH-USP)
São Paulo, Brazil
sarajane@usp.br

Abstract

The dynamic and non-deterministic nature of large language models (LLMs) complicates conversational agent evaluation, particularly when assessing pre-deployment configurations under realistic conditions. To address this, a digital-twin-inspired architecture is proposed to monitor interactions and fork ongoing dialogues, enabling what-if analyses of alternative agent configurations using observed conversational contexts. To assess technical feasibility, we conducted an exploratory experiment using 3,996 simulated dialogues as a production proxy, generating 949 evaluation dialogues across four twin agents with varying parameters. The resulting interaction traces were analyzed with process mining techniques, showing the feasibility of the proposed architecture and its potential to support comparative evaluation of conversational agent modifications before deployment.

Keywords

LLM-based Conversational Agents, Digital Twins, LLM Testing, LLM Evaluation, Process Mining

1 Introduction

Large Language Models (LLM) have expanded the capabilities of conversational agents, enabling them to support complex tasks and business processes [4, 13, 21]. However, agents evaluation are more challenging, since interactions are inherently dynamic and non-deterministic, making it difficult for a single benchmark or evaluation strategy to fully capture system performance [28]. Furthermore, assessing alternative agent configurations (e.g., prompts, models) prior to deployment adds up to the evaluation challenge. While offline benchmarks and laboratory experiments provide valuable insights, they may not adequately represent the contextual and behavioral characteristics of real-world interactions.

Digital Twins (DTs) have been adopted across varied domains to support monitoring, prediction, and optimization [7]. DTs are generally defined as digital representations of models and data from a real-world counterpart to support analysis and decisions [3, 6]. Inspired by runtime-oriented DT principles, this work proposes a DT-based architecture for the pre-deployment evaluation of LLM conversational agents. The system continuously monitors live interactions and branches ongoing dialogues under specific conditions,

enabling alternative configurations to be simulated and compared against the original agent using realistic operational contexts.

The remainder of the paper is organized as follows: Section 2 reviews theoretical background and related work; Section 3 presents the DT-inspired architecture; Section 4 discusses an exploratory experiment; Section 5 concludes the paper.

2 Theoretical Background

2.1 LLM-based Chatbots Monitoring

A conversational agent is a software system designed to interact with users through natural language, either via text or voice-based communication [1, 4, 15]. Conversational agents encompass a broad range of systems, including virtual assistants, task-oriented agents, and chatbots, and may support different goals and domains [1, 4]. Chatbots represent a common type of conversational agent, typically characterized by text-based interactions with users [1].

Recent advances in LLMs have established them as a foundation of modern conversational systems, expanding their capabilities and adoption while supporting increasingly natural, context- and task-aware interactions [4, 21]. To address limitations related to static knowledge and factual inaccuracies, these systems are often combined with Retrieval-Augmented Generation (RAG), which incorporates information from external sources into the generation process, enabling more accurate, up-to-date, and contextually grounded interactions [13].

However, the dynamic and non-deterministic nature of conversational interactions complicates evaluation. No single benchmark adequately captures system performance [28], particularly for LLM agents executing multi-step reasoning and tool use. Multiple evaluation perspectives have emerged: task-oriented approaches assess goal completion and policy compliance [27], process-oriented methods analyze interaction patterns from conversational logs [23], and observability strategies monitor lifecycle behavior and runtime metrics to support optimization [5, 24]. These perspectives highlight the multifaceted nature of conversational agent evaluation and the need to align monitoring strategies with analytical objectives.

2.2 Digital Twins

Digital Twins have been applied in domains such as manufacturing, healthcare, autonomous driving, and smart infrastructure, supporting activities including system analysis, monitoring, prediction,

control, and optimization [7]. Despite their growing adoption, DTs are defined in different ways in the literature, ranging from broad notions of digital representations to more restrictive or technology-specific conceptualizations [3, 7]. According to Dalibor et al., a DT comprises models representing the system of interest, digital shadows containing contextual data about its state and history, and services that leverage these artifacts to support analysis and decisions with respect to the original system [6]. Such services may include simulation and prediction capabilities to enable the evaluation of alternative configurations and optimization strategies. Building on this definition, we adopt a runtime-oriented perspective of Brockhoff et al. [3], which defines DTs as active software systems that continuously use data and computational models to understand, predict, and optimize the behavior of the represented systems [3]. Under this view, DTs continuously reflect the represented system and support data-driven reasoning at runtime.

2.3 Related Work

Recent research has explored the integration of LLMs and DTs. Most studies employ LLMs to enhance DT capabilities, such as decision support, prediction, and optimization in domains including robotics [22], industrial systems [14], and computer networks [10].

DTs have also been applied to conversational contexts. Examples include clinician DTs that emulate provider-patient communication [19], human-centered digital persona twins for training environments [2], and patient DTs that support medical history-taking and diagnostic reasoning [18]. In these works, LLMs are used to construct or operate the DT itself, enabling conversational behavior.

Beyond conversational applications, DTs have been extended to abstract domains, including business processes [9, 20]. DT architectures have also been proposed to support process monitoring, simulation, and improvement through data-driven representations of operational behavior [11, 26]. These studies demonstrate the applicability of DTs beyond traditional physical assets and their potential for analyzing complex socio-technical systems.

Despite the growing body of work combining DTs and LLMs, existing studies primarily employ LLMs as components of DTs or apply DT concepts to process and organizational analysis. Limited attention has been given to DT-inspired approaches for evaluating modifications to LLM-based conversational agents under realistic interaction conditions before deployment.

3 Digital Twin-Inspired Evaluation Approach

This work proposes a DT-inspired approach for the pre-deployment evaluation of modifications to LLM-based conversational agents. To implement such an approach, a software architecture was designed based on functional and non-functional requirements (Table 1). These requirements capture the capabilities needed to support conversational forking and comparative evaluation, as well as engineering concerns related to scalability, modularity, and reproducibility.

Figure 1 presents the proposed DT-inspired evaluation approach and the interaction flow among its main components. Conversational interactions are monitored and analyzed to identify predefined triggering conditions that may initiate the evaluation of alternative agent configurations through conversational branches.

Table 1: Requirements of the proposed approach. “FR”: functional requirements; “NFR”: non-functional requirements.

Type	Requirement
FR-1	Support conversational interactions between users and conversational agents.
FR-2	Analyze conversational interactions according to configurable monitoring criteria.
FR-3	Detect predefined triggering conditions (e.g. touchpoint occurrence in conversations) and instantiate conversational branches for evaluation purposes.
FR-4	Collect and store interaction data from both original and branched conversations for subsequent analysis.
NFR-1	Support near real-time operation to enable runtime monitoring and branching.
NFR-2	Promote modularity and loose coupling among architectural components.
NFR-3	Support the execution of multiple simultaneous conversational branches.
NFR-4	Facilitate reproducibility, extensibility, and experimentation.

The proposed approach comprises three main components: a conversational system under observation, a monitoring engine, and a fork engine. The conversational system corresponds to the operational agent whose interactions are continuously observed and serve as the basis for subsequent evaluations. The monitoring engine processes interactions and generates structured signals that can be evaluated against pre-configured triggering conditions. For example, a triggering condition may associate a specific touchpoint with one or more alternative twin configurations to be evaluated. When a condition is satisfied, the fork engine instantiates one or more conversational twins configured with alternative settings and continues the interaction from the selected conversational state.

The resulting forked dialogues are collected together with the original conversations, enabling comparative analyses between the operational agent and alternative configurations. By using observed interactions as the starting point for conversational forking, the approach preserves contextual characteristics of real-world ongoing conversations and evaluates alternative configurations from equivalent conversational situations rather than artificial test scenarios. Since the proposed approach is designed to generate conversations from alternative configurations, it requires the creation of digital twins of real users. In this sense, the architecture must incorporate LLM-based user personas capable of playing, as accurately as possible, the role of the user participating in the monitored conversation. For example, user personas may be calibrated against historical interaction data to better approximate observed user behavior.

The approach is agnostic to the techniques used for monitoring, trigger detection, user-persona simulation, and conversational analysis, supporting objective-specific implementations.

4 Exploratory Experiment

An exploratory experiment was conducted with a customer-service conversational agent in the banking domain to demonstrate how the proposed approach enables the comparison of alternative agent

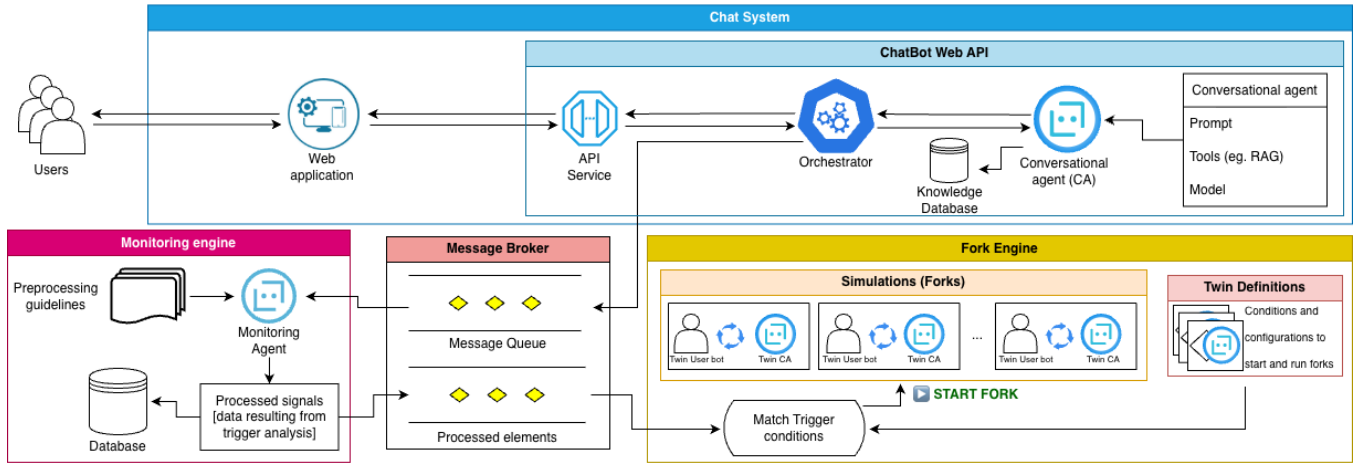


Figure 1: Proposed Digital Twin-inspired evaluation approach. Interactions from an operational conversational agent are monitored and analyzed. When predefined triggering conditions are satisfied, conversational branches are instantiated using twin agents configured with alternative settings, enabling comparative evaluation before deployment.

configurations from equivalent conversational situations derived from real-world ongoing customer-agent interactions. To emulate this scenario, a simulated environment generated a continuous stream of customer-agent dialogues.

4.1 Experiment Setup

This section details the components and configurations instantiated to operationalize the proposed approach in the experiment.

4.1.1 Operational Environment Simulation. Since the proposed approach relies on monitoring interactions within a conversational system, the experiment used a simulated operational environment to represent a system under observation. Following Silva et al. [23], the environment reproduced a customer-service chatbot for a fictitious Brazilian financial institution, with all interactions and experimental artifacts specified in Brazilian Portuguese.

The simulated environment comprised an LLM-based chatbot interacting with LLM-based customer personas. It was implemented in Python using LangChain and GPT-4.1. The chatbot supported requests concerning three credit card products through a RAG mechanism connected to a knowledge base constructed from publicly available information provided by Brazilian financial institutions.

The chatbot was implemented as a graph-based RAG workflow. Customer queries were reformulated to improve retrieval, relevant documents were selected from the knowledge base, and responses were generated based on the retrieved evidence or, when no sufficient evidence was found, through a fallback strategy.

Customer behavior was generated by combining predefined profiles and service objectives. Operational conditions were simulated through scenario configurations and persona characteristics designed to elicit dialogues of varying complexity.¹ A total of 3,996 dialogues were simulated as the Monitoring Engine’s interaction stream throughout the experiment and served as input to the Fork Engine’s conversational forking mechanism.

¹The complete simulation configuration is available in the project’s replication package.

4.1.2 Analysis Strategy. Traditional agent evaluation often relies on aggregate metrics (e.g., end-to-end task completion rate or average latency), which mask turn-level dynamics. Process Mining complements these metrics by mapping dialogue trajectories to control-flow graphs. This enables granular inspection of behavioral loops, redundant turns, dynamic pathing changes, and structural bottlenecks introduced when altering underlying models or retrieval setups. In this sense, the analysis performed in this experiment was based on Process Mining, which comprises a set of techniques that extract knowledge from event logs to discover, analyze, and compare behavioral patterns observed during process execution [25]. In this context, an event log is composed of cases, representing individual process instances, and events, representing activities executed within each case. To apply this perspective to conversational agents, each dialogue was interpreted as a process instance (or cases) and messages as observed events within each dialogue. Following Silva et al. [23], conversational interactions were mapped to event logs, enabling the use of Process Mining techniques to analyze and compare conversational behavior patterns [23]. More specifically, each message was mapped to an event through a predefined set of conversational touchpoints. This representation enables the comparison of alternative conversational agent configurations in terms of the behavioral patterns that emerge from the dialogues.

4.1.3 Monitoring Engine. The Monitoring Engine was responsible for transforming messages exchanged during dialogues into structured signals (touchpoints organized as events in an event log) suitable for trigger detection and subsequent process mining analysis. The adopted touchpoint set comprised 12 touchpoints describing customer messages and 12 touchpoints for chatbot messages. Each message exchanged during a dialogue was mapped to a single touchpoint using an ensemble of multiple LLMs with majority voting. In cases of ties, precedence was given to predictions produced by models with a larger number of parameters. The ensemble combined

the classifications produced by GPT-5.4 mini, DeepSeek-R1-14B (using Q6_K_L quantization)², Qwen3-14B (using 8 bit quantization)³, Gemma-4-E4B⁴, and Qwen3-8B⁵, in this order of precedence. The resulting touchpoint event logs were used both for Process Mining analyses and for evaluating trigger conditions in the Fork Engine.

4.1.4 Fork Engine. The Fork Engine was responsible for evaluating the triggering conditions defined for the experiment and instantiating dialogues fork whenever such conditions were satisfied. When a trigger was detected, the engine retrieved the conversational context associated with the original dialogue and instantiated one or more conversational twins. The forked dialogues then continued independently from the original interaction while preserving the conversational state observed at the triggering point.

Herein, the selected trigger was the touchpoint “*Direct Request of Human Agent*” (the original “*Solicitação Direta de Humano*” in Brazilian Portuguese), which represents situations in which customers explicitly request to be transferred to a human attendant. This trigger was selected because such requests often reveal limitations in the conversational agent’s ability to satisfy customer needs, making them suitable for evaluating alternative agent configurations. All configurations relied on knowledge bases covering the same three credit card products. Four conversational twins corresponding to the combinations of two language models and two retrieval strategies were evaluated:

- **GPT Single-RAG Twin (Twin 1):** *GPT-4.1* with a *Single-RAG* retrieval strategy – the knowledge base is exposed as a single retrieval tool available to the conversational agent. The agent autonomously decides whether and when to invoke retrieval based on its internal reasoning, following the Agentic-RAG paradigm [17]. This configuration serves as the reference implementation for the experiment, providing a flexible retrieval strategy in which external knowledge is accessed only when required, potentially reducing response latency and token consumption.
- **GPT Triple-RAG Twin (Twin 2):** *GPT-4.1* with a *Triple-RAG* retrieval strategy – the knowledge base was partitioned into three specialized retrieval tools, one for each credit card product.
- **DpSk Single-RAG Twin (Twin 3):** *DeepSeek-R1-14B* with the *Single-RAG* retrieval strategy – enabling evaluation of a smaller model as a replacement for the original.
- **DpSk Triple-RAG Twin (Twin 4):** *DeepSeek-R1-14B* with the *Triple-RAG* retrieval strategy, – combining both modifications.

The evaluated twins follow a 2×2 factorial design combining two language models and two retrieval strategies, enabling the isolated and combined effects of both factors to be analyzed. Conversational forking was limited to the first trigger occurrence in each dialogue to evaluate whether alternative configurations could avoid or postpone the initial human-assistance request.

4.2 Results

The simulated operational environment generated 3,996 dialogues comprising 36,315 messages. The Monitoring Engine processed all messages, producing the same number of touchpoints through the ensemble-based classification strategy. Among the identified touchpoints, 356 matched the selected trigger condition. Considering only the first trigger occurrence per dialogue for conversational fork, 238 dialogues were eligible for twin instantiation. Each eligible dialogue generated one instance of each of the four conversational twins, yielding 952 expected forked dialogues. Of these, 949 were successfully generated,⁶ producing 13,650 messages. The Monitoring Engine then processed these dialogues using the ensemble-based classification strategy to derive the touchpoints.⁷

The results are presented from three complementary perspectives. First, the event logs are characterized using general process mining metrics. Next, the dialogues are analyzed from a control-flow perspective through visual representations of the conversational process. Finally, trigger frequency and position are examined to characterize behavioral differences across configurations. Together, these analyses assess whether the proposed Digital Twin-inspired approach can reveal differences among alternative conversational agent configurations.

4.2.1 Event Log Characterization. Table 2 summarizes the main characteristics of the event logs generated by the Original dialogue log and by each conversational twin.

Table 2: Characteristics of the event logs. The value in parentheses indicates the difference between the twin and the original; events = touchpoint events.

	#Events	Events per Dialogue	Dialogue Variants
Original	2,562	10.76	214
Twin (1)	2,849 (+11.2%)	11.97 (+11.2%)	232 (+8.4%)
Twin (2)	2,811 (+9.7%)	11.86 (+10.2%)	232 (+8.4%)
Twin (3)	4,020 (+56.9%)	16.96 (+57.6%)	235 (+9.8%)
Twin (4)	3,970 (+55.0%)	16.75 (+55.6%)	236 (+10.3%)

The event logs show clear differences in trace length and behavioral variability. The original chatbot produced the most compact process behavior, with an average of 10.76 touchpoints per dialogue and 214 dialogue variants. GPT based twins (Twin (1) and Twin (2)) remained relatively close to the original, suggesting that partitioning the knowledge base into multiple retrieval tools did not substantially alter the overall conversational behavior under the evaluated scenarios. By contrast, DeepSeek based twins (Twin (3) and Twin (4)) produced substantially longer traces, with approximately 16.8 to 17.0 touchpoints per dialogue, representing an increase of about 56% to 58% over the original. However, their number of dialogue variants increased only modestly. This indicates that replacing the language model affected trace execution more than it

⁶Three simulations terminated unsuccessfully due to silent execution failures.

⁷The ensemble-based classification achieved Fleiss’ Kappa coefficients of 0.58 for the original dialogues and 0.42 for the twin-generated dialogues, indicating moderate agreement according to Landis and Koch [16]. This level of agreement is consistent with ensemble design principles: voting-based ensembles are useful precisely when individual classifiers are not fully redundant [8, 12].

²<https://huggingface.co/bartowski/DeepSeek-R1-Distill-Qwen-14B-GGUF>

³<https://huggingface.co/Qwen/Qwen3-14B-GGUF>

⁴<https://huggingface.co/google/gemma-4-E4B-it>

⁵<https://huggingface.co/Qwen/Qwen3-8B>

introduced a substantially larger set of distinct behavioral patterns. One possible explanation is that DeepSeek model intensified cyclic behavior already present in the original log, making such behavior more prominent rather than structurally new.

4.2.2 Conversational Control-Flow Analysis. This section presents an example of conversational control-flow analysis using process mining, supported by a well-known process discovery and visualization method: the Directly-Follows Graph (DFG). Combined with simplification filters, DFGs help extract the dominant behavior observed across process executions. Following this approach, we compared the DFG of the original chatbot with that of the twin that most differed from it, DpSk Triple-RAG Twin (Twin (4)), using the same 30% activity simplification (Figure 2).

The DFG comparison supports the quantitative results. The DpSk Triple-RAG twin preserves the same broad conversational backbone observed in the original chatbot, from chatbot presentation to user request, information response, and possible human-assistance request. However, the dominant dynamics of the process change substantially. The number of *"Request for Information or Action"* (originally *"Solicitação de Informação ou Ação"* in Brazilian Portuguese) events increases from 387 in the original log to 816 in the DpSk Triple-RAG log, whereas *Response to Information Request* increases only slightly, from 382 to 406. This indicates a concentration of behavior around request handling, without a proportional increase in information-response behavior. The DpSk Triple-RAG DFG also shows *Detected Intent Notification* as more structurally integrated into the dominant process flow. While this touchpoint is present in the original log, it does not appear in the simplified DFG. In the DpSk Triple-RAG log, it is more visibly connected to request-handling and human-assistance behavior, suggesting that the model tends to reprocess, reinterpret, or redirect the interaction instead of moving directly toward the requested handoff.

From a pre-deployment perspective, this can be a risk indicator. The candidate model does not introduce a different conversational structure, but it intensifies request-handling and intent-detection cycles around the handoff behavior. Unless complementary evidence shows better task resolution, this pattern should be treated as a warning sign against replacing the production chatbot with the DpSk Triple-RAG configuration.

4.2.3 Trigger Frequency and Position Analysis. Table 3 summarizes the frequency of selected trigger by reporting the number of dialogues containing at least one occurrence, the total number of occurrences, and the average number of occurrences per triggered dialogue. Figure 3 complements these results by showing, for each Original dialogue and its corresponding forked dialogue, how the position of the first trigger occurrence changes, allowing the trigger to be identified as preserved, postponed, anticipated, or absent after conversational forking. Since the four conversational twins exhibited very similar trigger-position distributions, only the Sankey diagram for Twin (4) is presented. This twin was selected because it produced the longest dialogue traces while remaining representative of the overall behavior observed across the evaluated twins.

Table 3 indicates that all conversational twins reduced both the number of dialogues containing human-assistance requests and the total number of trigger occurrences relative to the Original dialogues. Nevertheless, when the trigger occurred, it tended to be

Table 3: Human Assistance requests after conversational branching.

Twin	Dialogues with Trigger	Trigger Occurrences	Occurrences per Dialogue
Original	238	356	1.50
Twin (1)	101	175	1.73
Twin (2)	108	184	1.70
Twin (3)	100	236	2.36
Twin (4)	111	271	2.44

repeated more frequently. The Original log has 1.50 requests per triggered dialogue, while the GPT-based twins ranged from 1.7 to 1.73, and the DpSk-based twins ranged from 2.36 to 2.44. Therefore, although the evaluated configurations reduced trigger occurrences, they also tended to reinforce their persistence once the first request for human assistance had been made.

The Sankey diagram in Figure 3 complements the aggregate results reported in Table 3 by showing how the reduction in trigger frequency is distributed across dialogues. Rather than originating from a specific trigger position, dialogues from multiple original trigger locations converge to N/A, indicating that human-assistance requests were avoided throughout different stages of the conversation. For dialogues in which the trigger remained present, its first occurrence continued to concentrate at the same early positions observed in the Original log, predominantly at locations 4 and 6, with only a small number of later occurrences.

From a pre-deployment perspective, these results suggest different trade-offs among the evaluated twins. The GPT-based twins reduce the overall number of human-assistance requests while only modestly increasing their repetition within triggered dialogues. In contrast, the DpSk-based twins, particularly the DpSk Triple-RAG Twin, also reduce the number of escalated dialogues but substantially increase the repetition of human-assistance requests once the trigger occurs. This behavior may indicate that, although fewer conversations require escalation, requests for human assistance become more persistent after they first appear. Such conversational patterns deserve further investigation before deployment alongside complementary evidence on task completion and user experience.

4.2.4 Practical Implications and Trade-offs. In the evaluated scenarios, the twins using the smaller local model produced longer dialogue traces and more persistent human-assistance requests, revealing behavioral effects that should be considered alongside its potential cost and resource benefits. In contrast, changing from Single-RAG to Triple-RAG produced limited behavioral differences. These findings demonstrate how the proposed approach can support system designers in assessing configuration changes before deployment by considering both behavioral and technical factors.

4.2.5 Threats to Validity: The experiment used simulated customer dialogues generated from LLM personas. This controlled setup shows runtime monitoring and forking are technically feasible, but simulated personas may not reflect real human unpredictability, emotional nuance, or complex edge cases in production. Future evaluations in live environments are needed to validate these findings with real user interactions.

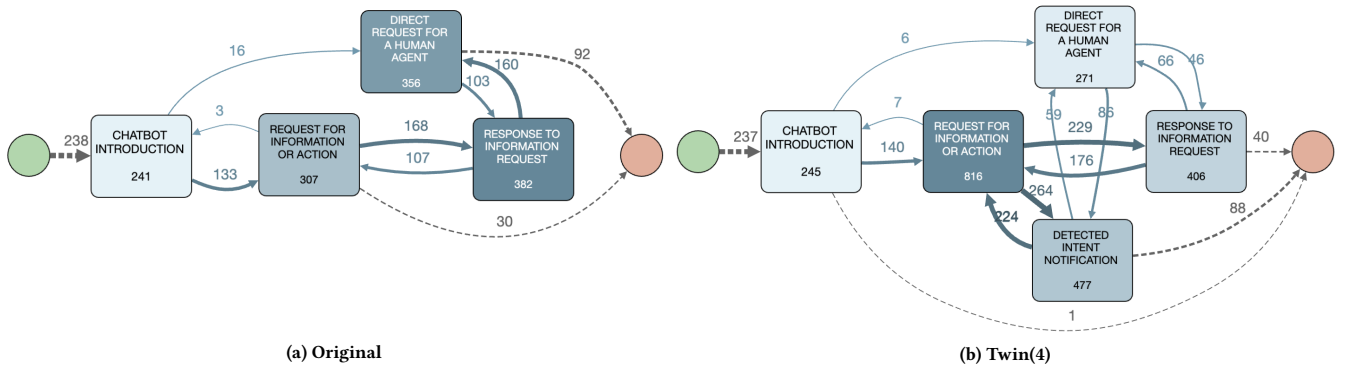


Figure 2: Directly-Follows Graphs for the dialogue logs of the Original (left) and DpSk Triple-RAG Twin (right) chatbots. The models were discovered with Apromore (Academic Alliance license), retaining the 30% most frequent activities.

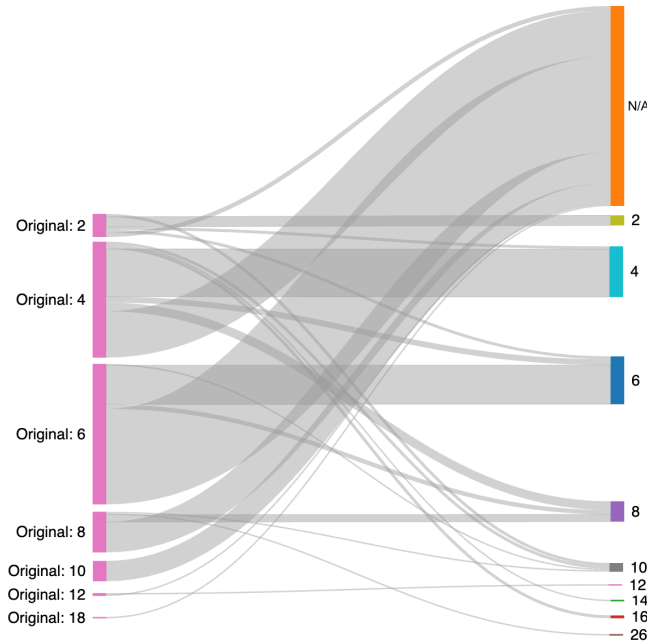


Figure 3: Mapping of the first trigger occurrence from the Original dialogues to their corresponding forked dialogues (Twin (4)). Location N/A indicates dialogues in which no human-assistance request occurred in the forked dialogue.

Regarding the event logs, even though mapping the messages to touchpoint labels reduces dependence on a single classifier, uncertainty remains because messages may be ambiguous and touchpoint boundaries are not strictly deterministic. As a consistency check, we applied conformance checking to classified logs from different twin configurations. A process model was discovered from one twin log, and a second twin log was replayed against it. The high fitness values obtained with a well-known method – token-replay [25] – indicate that the classified logs are structurally consistent from a process mining perspective. The fitness values obtained were: Twin

(1) $\times$ Twin (2) = 0.9985; Twin (3) $\times$ Twin (4) = 0.9977; Twin (3) $\times$ Twin (2) = 0.9956; Original $\times$ Twin (4) = 0.9921.

These results do not prove the semantic correctness of each message-to-touchpoint assignment, since systematic errors could be consistently reproduced by the same classification procedure. However, they provide complementary evidence that the ensemble produced coherent event logs suitable for a comparative process analysis perspective. Alternative validation procedures should still be useful in this context. Nevertheless, the conformance results reduce the likelihood that the observed process differences are merely artifacts of arbitrary touchpoint assignments.

5 Conclusion

This paper presented a Digital Twin-inspired approach for the pre-deployment evaluation of modifications to LLM-based conversational agents. By monitoring ongoing interactions and instantiating conversational twins from predefined trigger conditions, the proposed architecture enables comparative evaluation from equivalent conversational contexts. The exploratory experiment showed that the proposed approach can identify and characterize behavioral differences among alternative conversational agent configurations through complementary process mining analyses.

Future work includes evaluating the proposed approach in real-world conversational systems, investigating additional monitoring strategies and trigger conditions, and exploring conversational analysis techniques beyond process mining.

Artifact Availability

The source code, datasets, logs, and experimental results are publicly available at https://github.com/c2d-usp/digital_twin_chatbots, and are distributed under the terms of the MIT License and the Creative Commons Attribution-ShareAlike 4.0 International License.

Acknowledgments

The authors thank the Programa de Bolsas Itaú (PBI) of the Centro de Ciência de Dados (C2D), supported by Itaú Unibanco S.A.

References

- [1] Merav Allouch, Amos Azaria, and Rina Azoulay. 2021. Conversational Agents: Goals, Technologies, Vision and Challenges. *Sensors* 21, 24 (2021), 48 pages.
- [2] Amir Reza Asadi. 2026. Building Conversational Information Sources as Human-Centered Digital Persona Twins for Probation Officer Training. In *Proc. Conf. Hum. Inf. Interact. Retr.* (USA). ACM, USA, 394–398.
- [3] Tobias Brockhoff, Malte Heithoff, István Koren, Judith Michael, Jérôme Pfeiffer, Bernhard Rumpe, Merih Seran Uysal, Wil M. P. Van Der Aalst, and Andreas Wortmann. 2021. Process Prediction with Digital Twins. In *ACM/IEEE Int. Conf. Model Driven Eng. Lang. Syst. Companion*. IEEE, Japan, 182–187.
- [4] Fabio Caffaro and Giuseppe Rizzo. 2024. Knowledge-Enhanced Conversational Agents. *J. Comput. Sci. Technol.* 39, 3 (2024), 585–609.
- [5] Xin Chen, Yan Li, and Xiaoming Wang. 2025. Design Principles and Guidelines for LLM Observability: Insights from Developers. In *Proc. Ext. Abstr. CHI Conf. Hum. Factors Comput. Syst.* ACM, USA, 9 pages.
- [6] Manuela Dalibor, Malte Heithoff, et al. 2022. Generating customized low-code development platforms for digital twins. *J. Comp. Lang.* 70 (2022), 101117.
- [7] Manuela Dalibor, Nico Jansen, Bernhard Rumpe, et al. 2022. A Cross-Domain Systematic Mapping Study on Software Engineering for Digital Twins. *J. Syst. Software* 193 (2022), 111361.
- [8] Thomas G. Dietterich. 2000. Ensemble Methods in Machine Learning. In *Multiple Classifier Systems*. Lecture Notes in Computer Science, Vol. 1857. Springer, Germany, 1–15.
- [9] Fabrizio Fornari, Ivan Compagnucci, Massimo Callisto De Donato, et al. 2025. Digital Twins of Business Processes: A Research Manifesto. *Internet of Things* 30 (2025), 101477.
- [10] Zhiqi Guo, Fengxiao Tang, Linfeng Luo, Ming Zhao, and Nei Kato. 2025. A Survey on Applications of Large Language Model-Driven Digital Twins for Intelligent Network Optimization. *IEEE Commun. Surv. Tutorials* 28 (2025), 3388–3411.
- [11] Gyunam Park. 2021. Realizing A Digital Twin of An Organization Using Action-oriented Process Mining. In *Int. Conf. Process Min.* IEEE, Eindhoven, Netherlands, 104–111.
- [12] J. Han, M. Kamber, and J. Pei. 2011. *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann Publishers Inc., USA.
- [13] Yizheng Huang and Jimmy X. Huang. 2026. A Survey on Retrieval-Augmented Text Generation for Large Language Models. *ACM Comput. Surv.* 58, 12 (2026), 38 pages.
- [14] Yijun Huang, Jihan Zhang, Xi Chen, Alan H. F. Lam, and Ben M. Chen. 2024. From Simulation to Prediction: Enhancing Digital Twins with Advanced Generative AI Technologies. In *IEEE Int. Conf. Control Autom.* IEEE, Iceland, 490–495.
- [15] Daniel Jurafsky and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed.). Stanford, USA.
- [16] J. Richard Landis and Gary G. Koch. 1977. An Application of Hierarchical Kappa-Type Statistics in the Assessment of Majority Agreement among Multiple Observers. *Biometrics* 33, 2 (1977), 363–374. doi:10.2307/2529786
- [17] LangChain. 2026. LangChain Documentation: Retrieval. <https://docs.langchain.com/oss/python/langchain/retrieval>
- [18] Jung-Hyun Lee, Eunhee Choi, Sergio L. Angulo, Robert A. McDougal, and William W. Lytton. 2025. Neurological history both twinned and queried by generative artificial intelligence. *Frontiers in Medicine* 11 (2025), 9 pages.
- [19] Pengze Li, Yutong Hu, Jianfu Li, Garit Gemeinhardt, et al. 2026. Developing provider digital twins for personalized provider-patient communication via a RAG-based conversational framework. *Computational and Structural Biotechnology Journal* 32 (2026), 24–33.
- [20] Kalle Lyytinen, Barbara Weber, Markus C. Becker, and Brian T. Pentland. 2023. Digital twins of organization: implications for organization design. *J. Organ. Des.* 13, 3 (2023), 77–93.
- [21] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, et al. 2025. Large Language Models: A Survey. arXiv:2402.06196 [cs.CR]
- [22] Mehreen Naeem, Andrew Melnik, and Michael Beetz. 2025. Grounding Language Models with Semantic Digital Twins for Robotic Planning. arXiv:2506.16493 [cs].
- [23] Carlos Silva, Ana Maita, Gregorio Almeida Filho, Caio Silva, Gabriel Dimant, Renato Almeida, Guilherme Costa, Anna Araujo, Barbara Santana, Enio Blay, and Sarajane Peres. 2025. Evaluating LLM-Based Chatbots through Touchpoint-Driven Process Models. In *Anais do II Workshop sobre Bots na Engenharia de Software*. SBC, Recife, PE, Brazil, 1–10.
- [24] Twinkl Sisodia. 2026. AI Observability for Large Language Model Systems: A Multi-Layer Analysis of Monitoring Approaches from Confidence Calibration to Infrastructure Tracing. arXiv:2604.26152 [cs.SE]
- [25] W. M. P. van der Aalst. 2016. *Process Mining: Data Science in Action* (2 ed.). Springer, Berlin, Germany.
- [26] Francesco Vitale, Simone Guarino, Francesco Flammini, Luca Faramondi, Nicola Mazzocca, and Roberto Setola. 2025. Process Mining for Digital Twin Development of Industrial Cyber-Physical Systems. *IEEE Trans. Ind. Inf.* 21, 1 (2025), 866–875.
- [27] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *Int. Conf. Learn. Represent.* Curran Associates, Singapore, 53 pages.
- [28] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2026. Survey on Evaluation of LLM-based Agents. arXiv:2503.16416 [cs.AI].